ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

Đồ án Hệ thống Nhúng Thời gian thực

Internet vạn vật

LỚP: TN01

Giảng viên: Lê Trọng Nhân

STT	Họ và tên	MSSV	Tên lớp	Tên ngành
1	Nguyễn Lưu Khánh Trình	2313638	TN01	Kỹ thuật máy tính
2	Trần Minh Trí	2313626	TN01	Kỹ thuật máy tính

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 4 NĂM 2026

Mục lục

Tóm tắt **3**
Lời cảm ơn **4**
1 Giới thiệu **7**
1.1 Tổng quan đề tài . . . 7
1.2 Các nhiệm vụ và yêu cầu của đồ án . . . 7
1.3 Phạm vi và giới hạn . . . 7
1.4 Bố cục báo cáo . . . 8
2 Khái niệm hệ thống và cơ sở lý thuyết **9**
2.1 Giới thiệu . . . 9
2.2 Nền tảng IoT . . . 9
2.2.1 Thông số kỹ thuật chính . . . 9
2.2.2 Lý do lựa chọn ESP32 . . . 9
2.2.3 Các tính năng liên quan đến đồ án . . . 10
2.2.4 So sánh với các nền tảng thay thế . . . 10
2.3 Cơ sở về RTOS . . . 10
2.3.1 Khái niệm cốt lõi . . . 10
2.3.2 Ưu điểm của FreeRTOS . . . 11
2.3.3 Vai trò của FreeRTOS trong đồ án . . . 11
2.3.4 Kiến trúc tác vụ sử dụng trong đồ án . . . 12
2.3.5 So sánh với thiết kế firmware không dùng RTOS . . . 12
2.4 Cơ sở về TinyML . . . 12
2.4.1 Khái niệm cốt lõi . . . 12
2.4.2 Ưu điểm của TinyML . . . 13
2.4.3 Quy trình phát triển TinyML . . . 13
2.4.4 Ứng dụng TinyML trong đồ án . . . 13
2.4.5 Kiến trúc suy luận trong đồ án . . . 14
2.4.6 TinyML trên thiết bị so với ML trên đám mây . . . 14
2.4.7 Công cụ và thư viện hỗ trợ . . . 14
2.4.8 Các thách thức thực tế . . . 15
2.5 Cơ sở về CoreIoT . . . 15
2.6 Tóm tắt chương . . . 15
3 Thiết kế và triển khai hệ thống **16**
3.1 Giới thiệu . . . 16
3.2 Nhiệm vụ 1: Nhấp nháy LED theo nhiệt độ . . . 16
3.2.1 Nguyên lý hoạt động . . . 16
3.2.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore . . . 16
3.2.3 Sơ đồ máy trạng thái (FSM) . . . 17
3.3 Nhiệm vụ 2: Điều khiển NeoPixel theo độ ẩm . . . 17
3.3.1 Nguyên lý hoạt động . . . 17
3.3.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore . . . 18
3.3.3 Sơ đồ máy trạng thái (FSM) . . . 19
3.4 Nhiệm vụ 3: Giám sát nhiệt độ và độ ẩm với LCD . . . 19
3.4.1 Nguyên lý hoạt động . . . 19
3.4.2 Cơ chế điều phối hệ thống bằng Semaphore . . . 20
3.4.3 Sơ đồ máy trạng thái (FSM) . . . 21
3.5 Nhiệm vụ 4: Máy chủ web ở chế độ Access Point . . . 21

3.6 Nhiệm vụ 5: Triển khai TinyML và đánh giá độ chính xác 21
3.6.1 Xây dựng tập dữ liệu 21
3.6.2 Huấn luyện các mô hình và đánh giá để chọn mô hình phù hợp nhất 22
3.6.3 Kiểm thử lại mô hình và nhúng xuống firmware 23
3.7 Nhiệm vụ 6: Gửi dữ liệu lên máy chủ đám mây CoreIoT 24
3.7.1 Nguyên lý hoạt động 24
3.7.2 Sơ đồ máy trạng thái (FSM) 25
3.8 Tổng quan kiến trúc 25
3.9 Tính mới và cải tiến 25
3.10 Tóm tắt chương 25
4 Đánh giá và kết quả thực nghiệm 26
4.1 Giới thiệu 26
4.2 Các kịch bản kiểm thử 26
4.3 Kết quả chức năng 26
4.4 Đánh giá TinyML 26
4.5 Thảo luận 26
4.6 Tóm tắt chương 26
5 Kết luận và thảo luận nhóm 27
5.1 Thảo luận và kết luận 27
5.2 Hướng phát triển trong tương lai 27
Tài liệu tham khảo 28
A Phụ lục A: Tổ chức nhóm 29
B Phụ lục B: Tài liệu bổ sung 30

Tóm tắt

[Viết một phần tóm tắt ngắn gọn: mục tiêu, phương pháp, kết quả chính và từ khóa.]

Lời cảm ơn

[Viết lời cảm ơn dành cho giảng viên, các thành viên nhóm và những tài liệu tham khảo hữu ích.]

Danh sách hình

1 Task 1 FSM 17
2 Task 2 FSM 19
3 Task 3 FSM 21
4 Task 6 FSM 25

Danh sách bảng

1 Giới thiệu

1.1 Tổng quan đề tài

Đồ án này xây dựng một hệ thống IoT thông minh dựa trên nền tảng ESP32 để giám sát và điều khiển môi trường theo thời gian thực. Hệ thống thu thập dữ liệu nhiệt độ và độ ẩm từ các cảm biến tích hợp, hiển thị kết quả qua giao diện web, đồng thời cho phép người dùng cấu hình Wi-Fi, theo dõi trạng thái thiết bị và điều khiển từ xa một số chức năng đã chọn. Bên cạnh đó, đồ án tích hợp mô hình Machine Learning/TinyML để phân tích dữ liệu cảm biến và đưa ra dự đoán hoặc khuyến nghị phù hợp với trạng thái môi trường quan sát được. Giải pháp đề xuất hướng đến tự động hóa, cải thiện trải nghiệm người dùng và vẫn phù hợp với giới hạn tài nguyên xử lý và bộ nhớ của vi điều khiển ESP32.

1.2 Các nhiệm vụ và yêu cầu của đồ án

Phần triển khai được tổ chức thành sáu nhiệm vụ chính:

1. **Nhiệm vụ 1: Nhấp nháy LED theo nhiệt độ** — điều khiển LED trạng thái nhấp nháy dựa trên các mức nhiệt độ đo được.
2. **Nhiệm vụ 2: Điều khiển NeoPixel theo độ ẩm** — thay đổi màu sắc/hành vi của NeoPixel dựa trên các khoảng độ ẩm.
3. **Nhiệm vụ 3: Giám sát nhiệt độ và độ ẩm với LCD** — đọc dữ liệu cảm biến liên tục và hiển thị kết quả thời gian thực trên giao diện LCD.
4. **Nhiệm vụ 4: Máy chủ web ở chế độ Access Point** — triển khai máy chủ web trên ESP32-S3 để cung cấp chức năng cấu hình và giám sát cục bộ qua chế độ AP.
5. **Nhiệm vụ 5: Triển khai TinyML và đánh giá độ chính xác** — đưa mô hình TinyML lên thiết bị và đánh giá chất lượng suy luận bằng các thước đo phù hợp.
6. **Nhiệm vụ 6: Gửi dữ liệu lên máy chủ đám mây CoreIoT** — truyền dữ liệu thu thập được và trạng thái thiết bị đến backend CoreIoT để giám sát từ xa.

Toàn bộ hoạt động lập trình, tích hợp và kiểm thử được thực hiện trên nền tảng **ESP32-S3** với **PlatformIO** làm môi trường phát triển chính.

1.3 Phạm vi và giới hạn

[Mô tả những nội dung mà báo cáo bao quát và những phần nằm ngoài phạm vi.]

1.4 Bố cục báo cáo

[Trình bày ngắn gọn cấu trúc các chương còn lại của báo cáo.]

2 Khái niệm hệ thống và cơ sở lý thuyết

2.1 Giới thiệu

[Giới thiệu khái niệm hệ thống và lý do cần trình bày phần cơ sở lý thuyết.]

2.2 Nền tảng IoT

ESP32 là họ vi điều khiển 32-bit do Espressif Systems phát triển, kế thừa từ ESP8266. Nền tảng này tích hợp kết nối Wi-Fi và Bluetooth/BLE, đồng thời cung cấp hiệu năng xử lý đủ tốt cho các ứng dụng IoT biên. Trong đồ án này, ESP32-S3 được dùng làm bộ điều khiển trung tâm cho các tác vụ cảm biến, xử lý cục bộ, truyền thông và tương tác với đám mây.

2.2.1 Thông số kỹ thuật chính

- **Processor:** Xtensa/RISC-V class 32-bit architecture (model-dependent), typically operated in the 160–240 MHz range.
- **Memory:** On-chip SRAM resources with external Flash support (commonly 4–16 MB, depending on board configuration).
- **Connectivity:** IEEE 802.11 b/g/n Wi-Fi and Bluetooth Low Energy.
- **Peripheral Interfaces:** GPIO, PWM, ADC, UART, SPI, and I^2C.
- **Power Features:** Support for low-power modes suitable for embedded and battery-aware IoT systems.
- **Operating Range:** Industrial-grade operating conditions for practical deployment scenarios.

2.2.2 Lý do lựa chọn ESP32

Nền tảng ESP32 được lựa chọn vì các lý do sau:

1. **Integrated wireless connectivity:** native Wi-Fi support simplifies device-to-cloud communication and local web control.
2. **Balanced performance:** sufficient processing capability to execute sensor pipelines, web services, and lightweight TinyML inference on one device.
3. **Practical memory capacity:** enough resources for buffering sensor data, serving web assets, and running compact inference models.
4. **Strong software ecosystem:** mature support through PlatformIO, Arduino-compatible libraries, and broad community documentation.

5. **Cost-effectiveness:** favorable hardware cost compared to other IoT platforms with similar connectivity features.
6. **Energy efficiency:** suitable for long-running embedded applications and power-constrained scenarios.

2.2.3 Các tính năng liên quan đến đồ án

Trong đồ án này, các khả năng sau của ESP32 được sử dụng trực tiếp:

- **I^{2}C interface:** communication with digital sensors for temperature and humidity acquisition.
- **GPIO/PWM:** control of status LEDs, NeoPixel behavior, and other actuators.
- **Web server support:** local monitoring and control in Access Point mode.
- **Flash file system support (e.g., LittleFS):** storage of static web resources (HTML/CSS/JS).
- **TinyML compatibility:** on-device inference for lightweight environmental state analysis.

2.2.4 So sánh với các nền tảng thay thế

So với các lựa chọn phổ biến như Arduino Uno, Arduino MKR WiFi hoặc các board thuộc họ Raspberry Pi Pico, ESP32 cân bằng tốt hơn giữa năng lực tính toán, kết nối không dây tích hợp, hỗ trợ phần mềm và chi phí triển khai. Nhờ đó, ESP32 đặc biệt phù hợp với một đồ án IoT toàn diện, kết hợp cảm biến, trí tuệ cục bộ, tương tác web và gửi dữ liệu lên đám mây.

2.3 Cơ sở về RTOS

FreeRTOS là một nhân hệ điều hành thời gian thực (RTOS) gọn nhẹ, mã nguồn mở, được thiết kế cho các hệ thống nhúng có tài nguyên hạn chế. Hệ điều hành này cung cấp một lớp trừu tượng có cấu trúc cho thực thi đồng thời, xử lý sự kiện và đồng bộ giữa các tác vụ mà không yêu cầu lập trình viên phải tự xây dựng cơ chế lập lịch phức tạp. Nhờ chi phí tài nguyên thấp và hệ sinh thái hỗ trợ trưởng thành, FreeRTOS được sử dụng rộng rãi trên các nền tảng vi điều khiển như ESP32, STM32 và ARM Cortex-M.

2.3.1 Khái niệm cốt lõi

- **Task:** an independent execution unit, typically implemented as an infinite loop with its own stack and runtime state.
- **Scheduler:** the kernel component that decides which task executes at each instant; FreeRTOS commonly uses preemptive priority-based scheduling.

- **Queue:** a thread-safe communication mechanism for passing messages or data between tasks.
- **Semaphore:** a synchronization primitive used for signaling events and controlling access to shared resources.
- **Mutex:** a specialized locking mechanism for protecting critical shared resources from concurrent access.

2.3.2 Ưu điểm của FreeRTOS

FreeRTOS mang lại nhiều lợi ích thực tiễn cho đồ án này:

1. **Multitasking support:** multiple functional components can run concurrently as independent tasks.
2. **Open-source flexibility:** the kernel is freely available and can be adapted to project-specific constraints.
3. **Low resource footprint:** suitable for microcontrollers with limited memory and compute resources.
4. **Modularity and scalability:** queue, semaphore, software timer, and synchronization features can be incrementally integrated.
5. **Strong platform support:** deeply integrated with ESP-IDF and well supported by the embedded systems community.
6. **Industrial reliability:** broadly validated in large-scale commercial embedded deployments.

2.3.3 Vai trò của FreeRTOS trong đồ án

Trong hệ thống IoT này, FreeRTOS được dùng để điều phối nhiều mô-đun có yêu cầu thời gian và truyền thông cao:

1. **Concurrent task management:** separate tasks handle web serving, periodic sensor sampling, TinyML inference, and LED/device control.
2. **Data synchronization:** semaphores protect shared sensor data paths, while queues support safe task-to-task data transfer.
3. **Priority-aware response:** web request handling can be assigned higher priority for responsive user interaction; sensing and inference tasks run at controlled medium priorities.
4. **Resource isolation:** each task has dedicated stack allocation, reducing memory interference and improving system stability.

2.3.4 Kiến trúc tác vụ sử dụng trong đồ án

Thiết kế phần mềm tuân theo mô hình tổ chức tác vụ nhiều lớp của FreeRTOS:

- **Task WebServer (high priority):** handles HTTP requests and local UI interactions.
- **Task Sensor Reader (medium priority):** acquires periodic temperature and humidity measurements.
- **Task TinyML (medium priority):** executes on-device inference for state prediction.
- **Task LED/Device Control:** updates LED/NeoPixel and related actuator outputs.
- **Task Wi-Fi Manager:** manages connectivity and reconnection logic.
- **Idle Task (lowest priority):** runs when no application task is ready.

2.3.5 So sánh với thiết kế firmware không dùng RTOS

Nếu không có RTOS, việc phát triển firmware thường phải dựa nhiều vào cơ chế ngắt và các máy trạng thái được quản lý thủ công, khiến hệ thống khó mở rộng và bảo trì khi độ phức tạp tăng lên. FreeRTOS khắc phục các hạn chế này bằng cách phân rã ứng dụng thành các tác vụ độc lập và giao việc lập lịch, đồng bộ cho nhân hệ điều hành, từ đó cải thiện khả năng bảo trì, tính dự đoán về thời gian và độ ổn định trước lỗi tranh chấp tài nguyên.

2.4 Cơ sở về TinyML

TinyML (Tiny Machine Learning) tập trung vào việc triển khai suy luận học máy trên các thiết bị nhúng có tài nguyên hạn chế như vi điều khiển và cảm biến thông minh. Thay vì chuyển luồng dữ liệu cảm biến thô lên máy chủ đám mây để xử lý AI, TinyML thực hiện dự đoán trực tiếp ở biên. Cách tiếp cận này giúp giảm độ trễ, giảm chi phí năng lượng cho truyền thông, tăng cường riêng tư và cho phép hệ thống tiếp tục hoạt động ngay cả khi mạng không ổn định.

2.4.1 Khái niệm cốt lõi

- **Inference:** using a pre-trained model to predict new input samples; TinyML primarily targets inference rather than on-device training.
- **Model quantization:** reducing model precision (e.g., from float32 to int8/int16) to decrease memory usage and improve runtime efficiency.
- **TensorFlow Lite / TensorFlow Lite Micro:** lightweight runtimes for deploying compact models on embedded and IoT hardware.
- **Feature extraction:** converting raw sensor readings into meaningful features suitable for model input.

- **Edge AI:** running AI computation at the data source rather than in remote cloud infrastructure.

2.4.2 Ưu điểm của TinyML

Trong đồ án này, TinyML mang lại một số lợi ích thực tiễn:

1. **Low latency:** predictions are produced locally in near real time.
2. **Energy efficiency:** reduced network transmission lowers power consumption.
3. **Data privacy:** sensitive raw measurements can remain on-device.
4. **Offline capability:** core inference continues even when cloud connectivity is unavailable.
5. **Compact deployment:** optimized models can run within limited memory and compute budgets.
6. **Infrastructure cost reduction:** less dependency on high-capacity cloud inference services.

2.4.3 Quy trình phát triển TinyML

Quy trình triển khai thường bao gồm các bước sau:

1. Train a model offline using Python-based ML frameworks and historical datasets.
2. Convert the trained model to a lightweight deployment format (e.g., `.tflite` or equivalent compact representation).
3. Apply optimization/quantization to reduce memory footprint and improve execution speed.
4. Embed model parameters into firmware (for example as a C header file).
5. Deploy to ESP32-S3 and evaluate runtime latency, memory usage, and prediction quality.

2.4.4 Ứng dụng TinyML trong đồ án

Trong hệ thống IoT này, TinyML được sử dụng để phân loại trạng thái môi trường từ dữ liệu cảm biến thời gian thực:

- **Input features:** temperature (°C) and humidity (%) from the DHT20 sensor.
- **Output classes:** environmental operating states (e.g., invalid state, standby, cooling required, dehumidification required, humidification required).
- **Model choice:** Gaussian Naive Bayes (GNB), trained on historical labeled pairs of temperature-humidity samples.

- **Firmware integration:** model parameters exported to `naive_bayes_model.h` and compiled into the ESP32-S3 application.

To reduce communication overhead, the system can publish compact inference results (state labels) instead of transmitting all raw samples continuously.

2.4.5 Kiến trúc suy luận trong đồ án

Luồng suy luận được tổ chức như sau:

1. Acquire temperature and humidity from DHT20.
2. Apply preprocessing/feature formatting.
3. Run Gaussian Naive Bayes inference on ESP32-S3.
4. Produce the environmental-state classification.
5. Publish results to local UI and cloud dashboard.

2.4.6 TinyML trên thiết bị so với ML trên đám mây

Criterion	On-Device TinyML	Cloud ML
Latency	Very low (typically milliseconds)	Higher due to communication and server round-trip
Energy usage	Lower communication overhead	Higher network usage for continuous data upload
Data privacy	Raw data can remain on-device	Raw data is usually transmitted to remote servers
Network independence	Can continue inference offline	Requires stable connectivity for inference
Scalability	Limited by device resources	Scales with cloud infrastructure capacity
Model update flow	Often requires firmware/model redeployment	Centralized updates are easier on server side

2.4.7 Công cụ và thư viện hỗ trợ

Các công cụ phổ biến cho quy trình làm việc theo hướng TinyML gồm TensorFlow Lite, TensorFlow Lite Micro, các pipeline huấn luyện dựa trên scikit-learn và bộ SDK/thư viện nhúng sẵn có trong hệ sinh thái ESP32.

2.4.8 Các thách thức thực tế

Những ràng buộc chính của TinyML khi triển khai trên thiết bị nhúng bao gồm giới hạn chặt chẽ về RAM/Flash, khả năng suy giảm độ chính xác sau khi lượng tử hóa mạnh, và việc gỡ lỗi trên phần cứng khó hơn nhiều so với môi trường máy tính để bàn hoặc đám mây.

2.5 Cơ sở về CoreIoT

[Trình bày ngắn gọn luồng làm việc trên đám mây CoreIoT, đường truyền dữ liệu và các thành phần giám sát.]

2.6 Tóm tắt chương

[Tóm tắt nội dung của chương này.]

3 Thiết kế và triển khai hệ thống

3.1 Giới thiệu

[Giới thiệu chương triển khai và quy trình thiết kế.]

3.2 Nhiệm vụ 1: Nhấp nháy LED theo nhiệt độ

[Mô tả logic điều khiển, điều kiện kích hoạt và cách sử dụng semaphore.]

3.2.1 Nguyên lý hoạt động

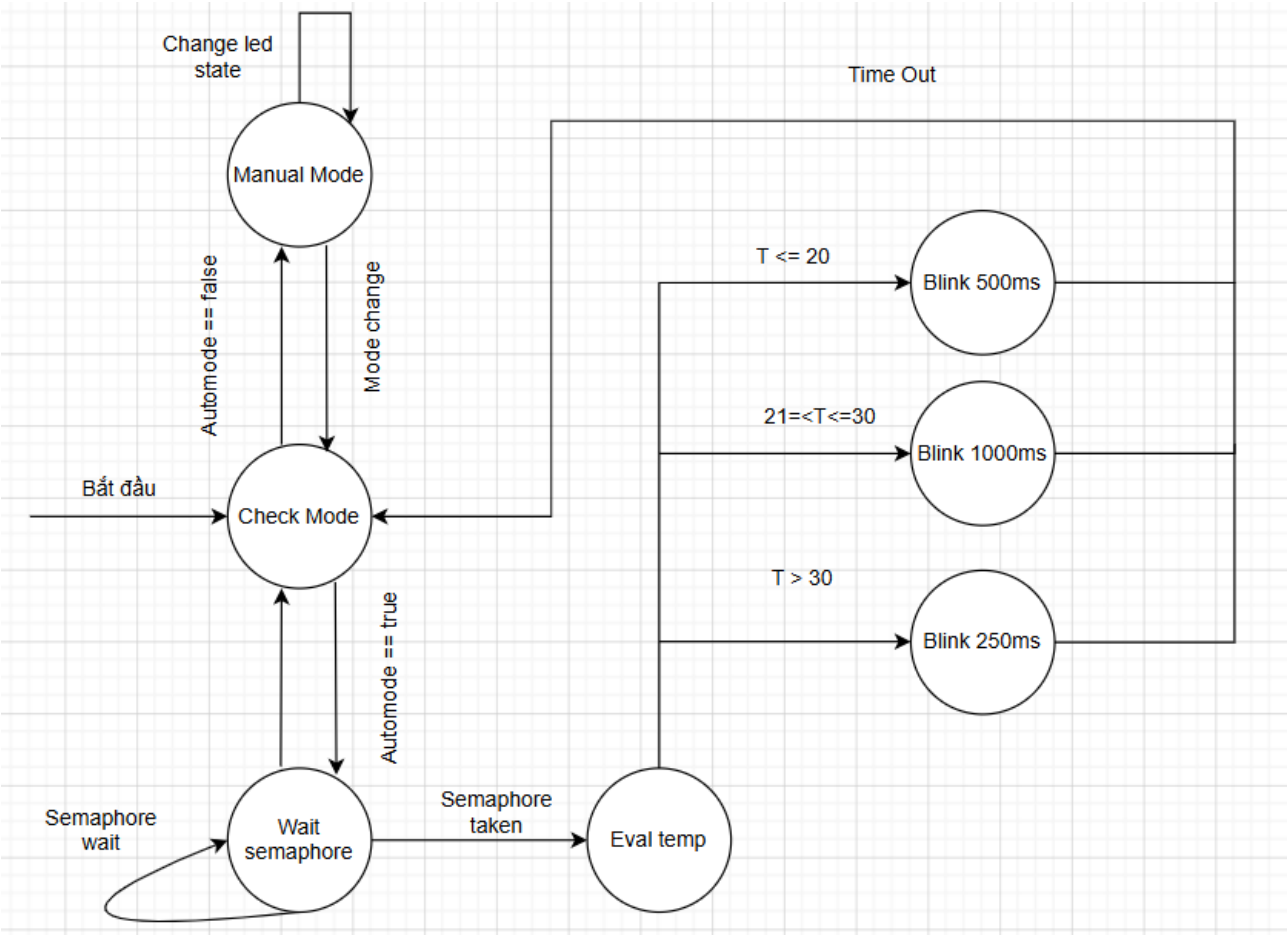
Tác vụ "Temperature-Driven LED Blinking"được thiết kế nhằm điều khiển trạng thái chớp tắt của đèn cảnh báo LED1 thông qua dữ liệu nhiệt độ môi trường hoặc sự can thiệp trực tiếp từ người dùng. Hệ thống hoạt động dựa trên cơ chế chuyển đổi linh hoạt giữa hai trạng thái chính: *Auto Mode* (Chế độ tự động) và *Manual Mode* (Chế độ thủ công).

- **Manual Mode (Chế độ thủ công):** Khi cờ `led_auto_mode` được đặt thành `false`, hệ thống sẽ bỏ qua các dữ liệu từ cảm biến. Trạng thái của LED1 lúc này được quyết định hoàn toàn bởi biến `led_manual_1`, cho phép phản hồi tức thời với các tín hiệu điều khiển từ xa qua Web Server hoặc nền tảng CoreIoT.
- **Auto Mode (Chế độ tự động):** Khi `led_auto_mode` là `true`, tác vụ sẽ chờ tín hiệu đồng bộ (Semaphore) từ quá trình đọc cảm biến. Tần số chớp tắt của LED được chia thành 3 dải nhiệt (T) như sau:
  - $T \leq 20°C$: Môi trường lạnh, chu kỳ nhấp nháy là 500ms.
  - $21°C \leq T \leq 30°C$: Môi trường ổn định, chu kỳ nhấp nháy chậm ở mức 1000ms.
  - $T > 30°C$: Cảnh báo nhiệt độ cao, chu kỳ nhấp nháy nhanh ở mức 250ms để thu hút sự chú ý.

3.2.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore

Được triển khai dưới dạng một FreeRTOS Task độc lập (`led_blinky`), hệ thống tránh được hiện tượng thắt cổ chai (blocking) khi xử lý các hàm `vTaskDelay`. Nhằm đảm bảo tính toàn vẹn dữ liệu (Thread-safe) giữa Task đọc cảm biến và Task điều khiển LED, một Binary Semaphore (`xBinarySemaphoreTemp_blinky`) đã được áp dụng. Task LED chỉ lấy giá trị nhiệt độ mới vào biến cục bộ `local_temp` khi chiếm được quyền kiểm soát Semaphore, qua đó ngăn chặn tuyệt đối lỗi "Data Race".

3.2.3 Sơ đồ máy trạng thái (FSM)



Hình 1: Task 1 FSM

3.3 Nhiệm vụ 2: Điều khiển NeoPixel theo độ ẩm

[Mô tả mẫu màu, các ngưỡng và cơ chế phối hợp dựa trên semaphore.]

3.3.1 Nguyên lý hoạt động

Tác vụ "Humidity and Network-Driven NeoPixel Control"chịu trách nhiệm điều khiển đèn LED RGB (NeoPixel) để phản ánh trực quan tình trạng độ ẩm không khí và trạng thái kết nối mạng của hệ thống. Tương tự như LED1, dải NeoPixel cũng hỗ trợ hai chế độ hoạt động: *Auto Mode* và *Manual Mode*.

- **Manual Mode (Chế độ thủ công):** Khi `led_auto_mode` bị tắt, màu sắc của NeoPixel được xác định bởi các giá trị `led_manual_r`, `led_manual_g`, `led_manual_b` và được tinh chỉnh độ sáng thông qua biến `led_brightness` (tính theo phần trăm). Sau khi cập nhật màu sắc, vòng lặp sẽ tạm nghỉ 150ms.

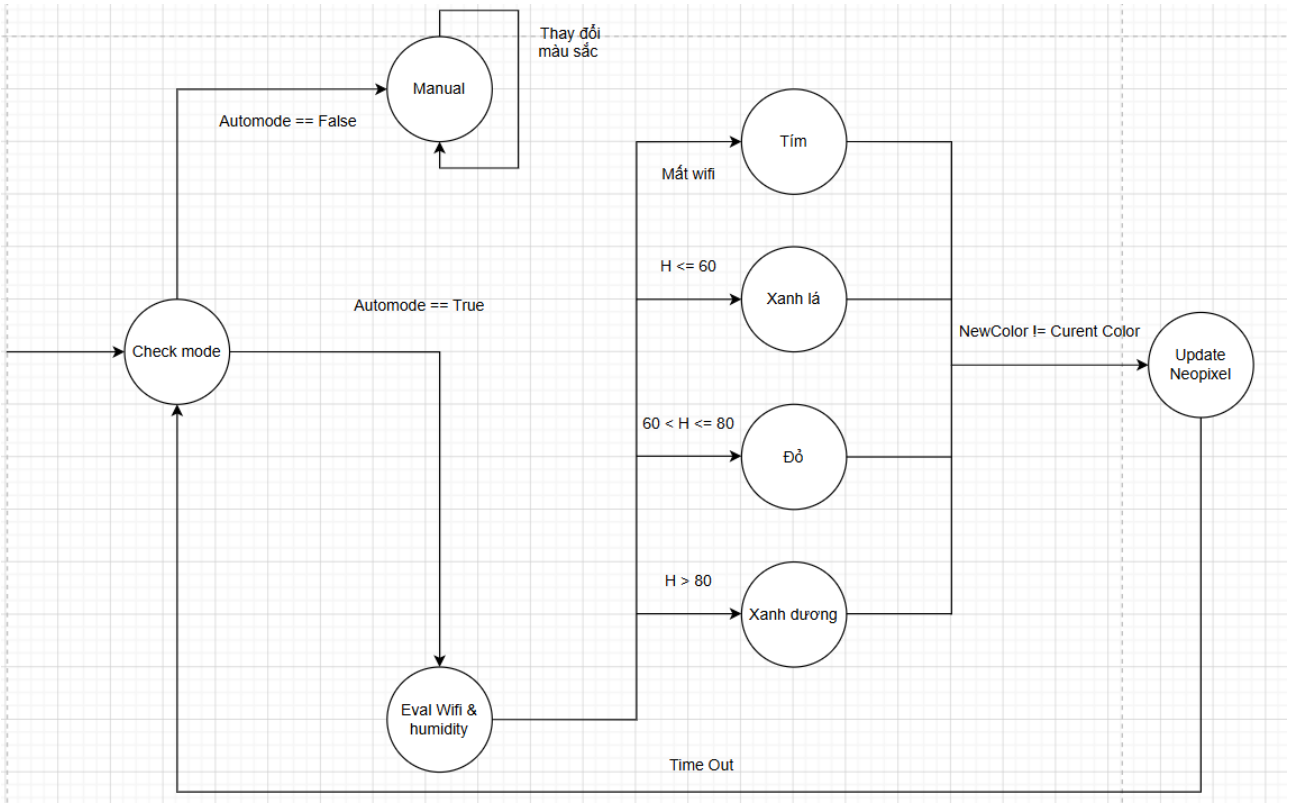
- **Auto Mode (Chế độ tự động):** Ở chế độ này, hệ thống sẽ ưu tiên kiểm tra trạng thái kết nối mạng, sau đó mới đánh giá dữ liệu độ ẩm (H) từ cảm biến. Các mức hiển thị màu sắc được quy định cụ thể như sau:
  - **Mất kết nối Wi-Fi** (`!isWifiConnected`): Đèn hiển thị màu Tím (Purple) cảnh báo mất mạng độc lập với dữ liệu cảm biến.
  - $H \leq 60\%$: Độ ẩm lý tưởng, đèn hiển thị màu Xanh lá (Green).
  - $60\% < H \leq 80\%$: Độ ẩm hơi cao, đèn hiển thị màu Đỏ (Red).
  - $H > 80\%$: Độ ẩm bão hòa, đèn hiển thị màu Xanh dương (Blue).

  Tác vụ chạy với chu kỳ delay là 300ms để đảm bảo cập nhật trạng thái liên tục mà không tiêu tốn tài nguyên hệ thống.

3.3.2 Cơ chế đồng bộ bằng FreeRTOS Semaphore

Tác vụ `neo_blinky` sử dụng hàm `xSemaphoreTake` để kiểm tra việc có hay không dữ liệu mới từ cảm biến. Khi nhận được Semaphore từ Task đọc cảm biến, biến cục bộ `local_hum` sẽ được cập nhật bằng giá trị mới nhất từ biến toàn cục `glob_humidity`. Điều này giúp vòng lặp của NeoPixel không bị chặn lại nếu cảm biến chưa đọc xong, duy trì khả năng cảnh báo mất Wi-Fi tức thời. Hệ thống cũng áp dụng kỹ thuật so sánh cờ `currentColor` và `newColor` để chỉ gọi hàm `strip.show()` khi thực sự có sự thay đổi màu sắc,

3.3.3 Sơ đồ máy trạng thái (FSM)



Hình 2: Task 2 FSM

3.4 Nhiệm vụ 3: Giám sát nhiệt độ và độ ẩm với LCD

[Giải thích việc lấy mẫu cảm biến định kỳ, định dạng dữ liệu và logic cập nhật LCD theo thời gian thực.]

3.4.1 Nguyên lý hoạt động

Tác vụ "Temperature and Humidity Monitoring"đóng vai trò là khối thu thập dữ liệu trung tâm (Producer) của toàn bộ hệ thống. Tác vụ này có trách nhiệm giao tiếp với cảm biến DHT20 qua chuẩn I2C để trích xuất nhiệt độ và độ ẩm, hiển thị trực quan lên màn hình LCD 16x2, đồng thời cung cấp dữ liệu đầu vào cho các tác vụ phân tích và cảnh báo khác.

Nguyên lý hoạt động được chia thành các bước kiểm duyệt và xử lý chặt chẽ:

- **Kiểm duyệt dữ liệu (Data Validation):** Sau mỗi lần đọc cảm biến, hệ thống sử dụng hàm `isnan()` để kiểm tra tính hợp lệ của dữ liệu. Nếu xuất hiện lỗi đọc (NaN), hệ thống sẽ in cảnh báo ra Serial, tạm dừng 1000ms và lặp lại quá trình đọc mà không làm thay đổi trạng thái của các biến toàn cục. Điều này giúp bảo vệ hệ thống khỏi các giá trị nhiễu hoặc rác.

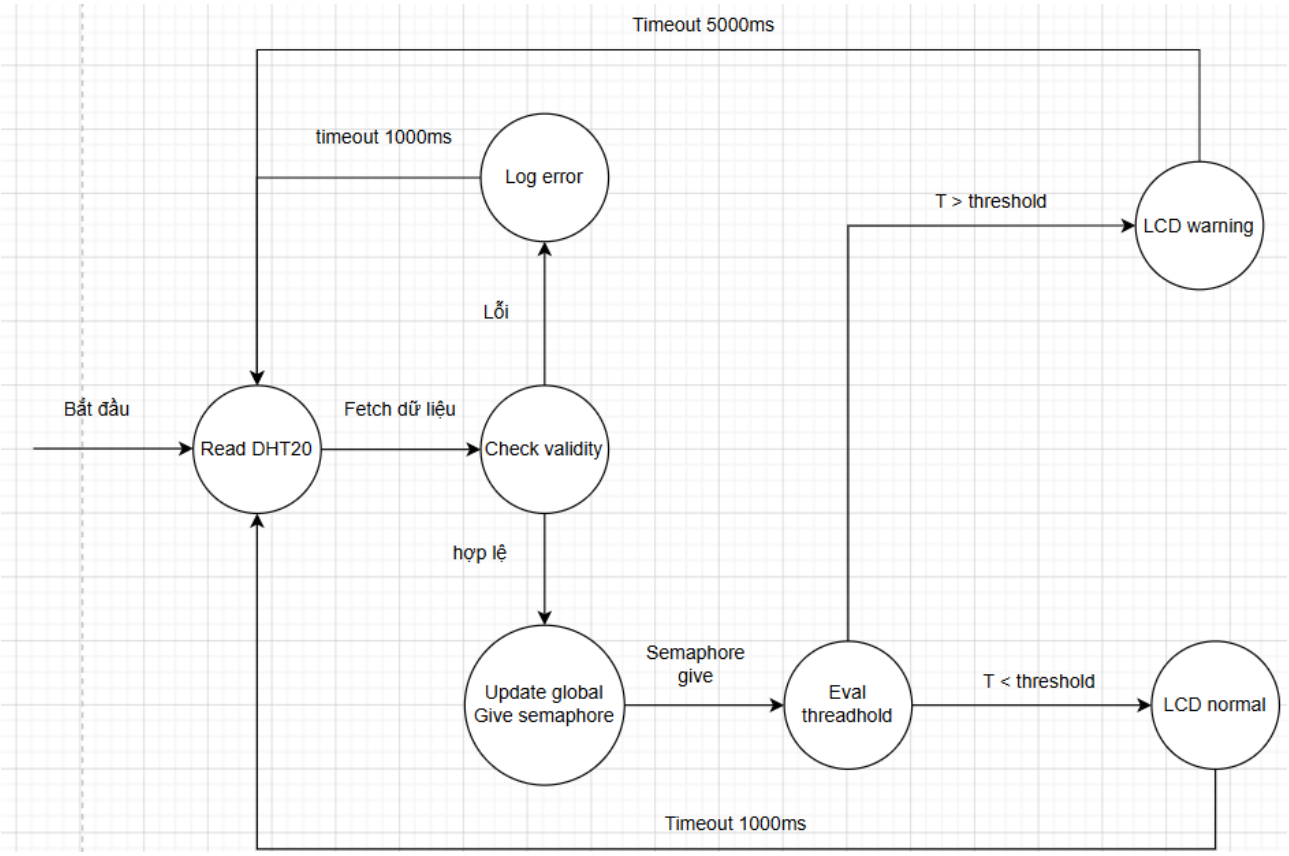
- **Hiển thị có điều kiện (Conditional Display):** Dữ liệu hợp lệ sẽ được so sánh với ngưỡng nhiệt độ `glob_temp_threshold` (có thể được cập nhật động từ nền tảng CoreIoT).
  - **Môi trường bình thường ($T \leq$ Threshold):** LCD hiển thị trực quan Độ ẩm (H) ở hàng trên và Nhiệt độ (T) ở hàng dưới. Hệ thống duy trì trạng thái này trong chu kỳ 1000ms trước khi cập nhật mẫu mới.
  - **Vượt ngưỡng cảnh báo ($T >$ Threshold):** LCD ngay lập tức chuyển sang chế độ báo động. Hàng dưới chốt cứng thông số nhiệt độ hiện tại kèm mức ngưỡng vi phạm. Hàng trên thực hiện hiệu ứng chớp tắt dòng chữ `"! WARNING TEMP !"`5 lần (chu kỳ 500ms sáng / 500ms tắt, tổng thời gian 5000ms) để thu hút tối đa sự chú ý của người dùng mà không làm gián đoạn hệ điều hành (nhờ `vTaskDelay`).

3.4.2 Cơ chế điều phối hệ thống bằng Semaphore

Khác với Task 1 và Task 2 đóng vai trò là "Consumer", Task giám sát nhiệt ẩm hoạt động như một "Producer". Ngay khi dữ liệu hợp lệ được ghi vào biến toàn cục `glob_temperature` và `glob_humidity`, tác vụ này lập tức phát tín hiệu đánh thức các phân hệ khác bằng cách giải phóng đồng thời 3 Binary Semaphore:

1. `xBinarySemaphoreTemp_blinky`: Kích hoạt tính toán lại tần số nháy LED1.
2. `xBinarySemaphoreTemp_neo`: Kích hoạt đánh giá màu sắc dải LED NeoPixel.
3. `xBinarySemaphoreTinyMLData`: Kích hoạt bộ nội suy Machine Learning (Naive Bayes) để dự đoán nhãn trạng thái môi trường.

3.4.3 Sơ đồ máy trạng thái (FSM)



Hình 3: Task 3 FSM

3.5 Nhiệm vụ 4: Máy chủ web ở chế độ Access Point

[Mô tả thiết lập chế độ AP, thiết kế các HTTP endpoint và xử lý giao diện web trên ESP32-S3.]

3.6 Nhiệm vụ 5: Triển khai TinyML và đánh giá độ chính xác

[Trình bày luồng tích hợp TinyML, quá trình suy luận trên thiết bị và các thước đo đánh giá.]

3.6.1 Xây dựng tập dữ liệu

Đầu tiên, nhóm nghiên cứu thu thập dữ liệu từ cảm biến nhiệt độ và độ ẩm trong môi trường thực tế. Trong đó file collect_form_log.py có nhiệm vụ đọc log cảm biến từ cổng Serial của ESP32 và chuẩn hóa thành tập dữ liệu thô phục vụ huấn luyện. Script được thiết kế với cơ chế parse nhiều tầng nhằm tăng khả năng tương thích với các định dạng log khác nhau, bao gồm JSON, chuỗi key-value T/H, và định dạng phân tách bằng dấu phẩy.
Mỗi mẫu hợp lệ được ghi ngay vào CSV kèm timestamp UTC, giúp đảm bảo tính truy vết của dữ liệu. Để tăng độ tin cậy, hệ thống tự khởi tạo header khi file mới, ghi nối dữ liệu ở các phiên thu thập khác nhau, và flush sau mỗi mẫu để hạn chế mất dữ liệu khi dừng đột ngột. Kết

quả của bước này là tập raw_data ổn định, làm đầu vào trực tiếp cho khâu gán nhãn và tạo tập train_ready trong pipeline ML. Tuy nhiên do nhóm nghiên cứu không thể thu thập data ở trạng thái nhiệt độ, độ ẩm ở mức quá cao hoặc quá thấp nên phải sử dụng các lệnh script khác để tạo thêm dữ liệu đảm bảo tính tổng quát hóa của tập dữ liệu.
Tiếp theo, nhóm tiến hành gán nhãn cho tập dữ liệu thô dựa trên các ngưỡng nhiệt độ và độ ẩm đã định nghĩa. Script extract_ready_data.py được phát triển để tự động hóa quá trình này, giúp đảm bảo tính nhất quán và giảm thiểu sai sót so với gán nhãn thủ công. Quy tắc gán nhán được thiết lập dựa trên các mức nhiệt độ và độ ẩm, ví dụ:

- Khi độ ẩm nằm ngoài khoảng [0-100%] hoặc nhiệt độ nằm ngoài khoảng [-40°C đến 100°C], mẫu được gán nhãn "Không hợp lệ".
- Khi nhiệt độ nhỏ hơn 24°C thì sẽ được gán nhãn là "Bật máy sưởi".
- Khi nhiệt độ lớn hơn 33°C và độ ẩm nằm trong khoảng [0-80%] thì sẽ được gán nhãn là "Bật điều hòa".
- Khi nhiệt độ lớn hơn 33°C và độ ẩm lớn hơn 80% thì sẽ được gán nhãn là "Bật máy hút ẩm".
- Còn lại sẽ được gán nhãn là "Chế độ bình thường".

Lúc này tập dữ liệu đã được gán nhãn sẽ được lưu vào file train_ready.csv, sẵn sàng cho bước huấn luyện mô hình ML.

3.6.2 Huấn luyện các mô hình và đánh giá để chọn mô hình phù hợp nhất

[Trình bày các mô hình đã thử nghiệm, tiêu chí đánh giá (độ chính xác, độ trễ, bộ nhớ) và lý do chọn mô hình cuối cùng.] Từ tập dữ liệu đã đươc gán nhãn, nhóm tiến hành huấn luyện nhiều mô hình ML khác nhau, bao gồm Decision Tree, Random Forest, SVM và một MLP đơn giản. Mỗi mô hình được đánh giá dựa trên các thước đo sau:

- Độ chính xác (Accuracy): Trên tập huấn luyện (Train) và tập kiểm thử (Test) để đo lường khả năng dự đoán và kiểm tra hiện tượng quá khớp (overfitting).
- Kích thước mô hình (Artifact Size): Đo bằng Kilobytes (KB).
- Thời gian suy luận (Inference Latency): Đo bằng mili-giây (ms/sample). Quyết định độ trễ của hệ thống khi phản hồi môi trường thực tế.

Mô hình Mạng Nơron Đa tầng (MLP)

Mô hình Mạng Nơ-ron Đa tầng (MLP) trong hệ thống được thiết kế chuyên biệt để giải quyết bài toán phân loại trạng thái thiết bị nhưng vẫn phải tuân thủ giới hạn tài nguyên khắt khe

của vi điều khiển ESP32. Kiến trúc mạng được khởi tạo qua thư viện Keras rất tinh gọn: bao gồm một lớp đầu vào nhận hai đặc trưng môi trường (Nhiệt độ và Độ ẩm), theo sau là hai lớp ẩn (hidden layers) mỏng với chỉ 16 nơ-ron mỗi lớp sử dụng hàm kích hoạt phi tuyến ReLU. Lớp đầu ra sử dụng hàm Softmax để xuất ra xác suất cho từng trạng thái của thiết bị (như bật điều hòa, chờ, hút ẩm...).

Để đảm bảo mô hình MLP có thể học được các mẫu hình (patterns) một cách hiệu quả, toàn bộ dữ liệu đầu vào được yêu cầu phải đi qua bước chuẩn hóa Z-score. Bước tiền xử lý này ép các giá trị nhiệt độ và độ ẩm về cùng một thang đo với trung bình bằng 0 và độ lệch chuẩn bằng 1, giúp thuật toán tối ưu hóa Adam (với learning rate là 1e-3) hội tụ nhanh và ổn định hơn. Trong pha huấn luyện, mô hình sử dụng hàm mất mát sparse_categorical_crossentropy cho nhãn phân loại dạng số nguyên. Bên cạnh đó, hệ thống cũng áp dụng chiến lược kiểm định chéo (validation split) bằng cách trích ra 20% dữ liệu từ tập huấn luyện để theo dõi sát sao độ chính xác qua từng chu kỳ học (epoch), giúp phát hiện sớm và phòng tránh hiện tượng học vẹt (overfitting).

Thay vì triển khai mô hình gốc dưới định dạng số thực 32bit (Float32) vốn đòi hỏi nhiều dung lượng và năng lực tính toán dấu phẩy động, hệ thống sử dụng TFLiteConverter để thực hiện lượng tử hóa toàn phần về số nguyên 8-bit (Full Integer INT8 Quantization). Để thực hiện điều này mà không làm suy giảm nghiêm trọng độ chính xác, một tập dữ liệu đại diện (representative_dataset) đã được đưa vào để bộ chuyển đổi đo lường và hiệu chỉnh dải giá trị động (dynamic range) của các nơ-ron. Kết quả là mô hình cuối cùng (_mlp_int8.tflite) có kích thước giảm đi khoảng 4 lần, thời gian suy luận (latency) giảm đáng kể và đặc biệt phù hợp để nhúng trực tiếp vào bộ nhớ Flash giới hạn của vi điều khiển ESP32.

Sau khi đánh giá các mô hình, nhóm quyết định sẽ sử dụng mô hình Navie Byess để nhúng xuống ESP32.

3.6.3 Kiểm thử lại mô hình và nhúng xuống firmware

Trước khi nhúng mô hình đã chọn xuống firmware, nhóm tiến hành kiểm thử lại trên tập dữ liệu test để đảm bảo độ chính xác và hiệu suất của mô hình. Ngoài nhóm còn chạy script python trong file test_navie_bayes.py để đưa vào các mẫu dữ liệu mới để kiểm tra độ chính xác của mô hình trên các dữ liệu chưa từng thấy trước đó.

3.7 Nhiệm vụ 6: Gửi dữ liệu lên máy chủ đám mây CoreIoT

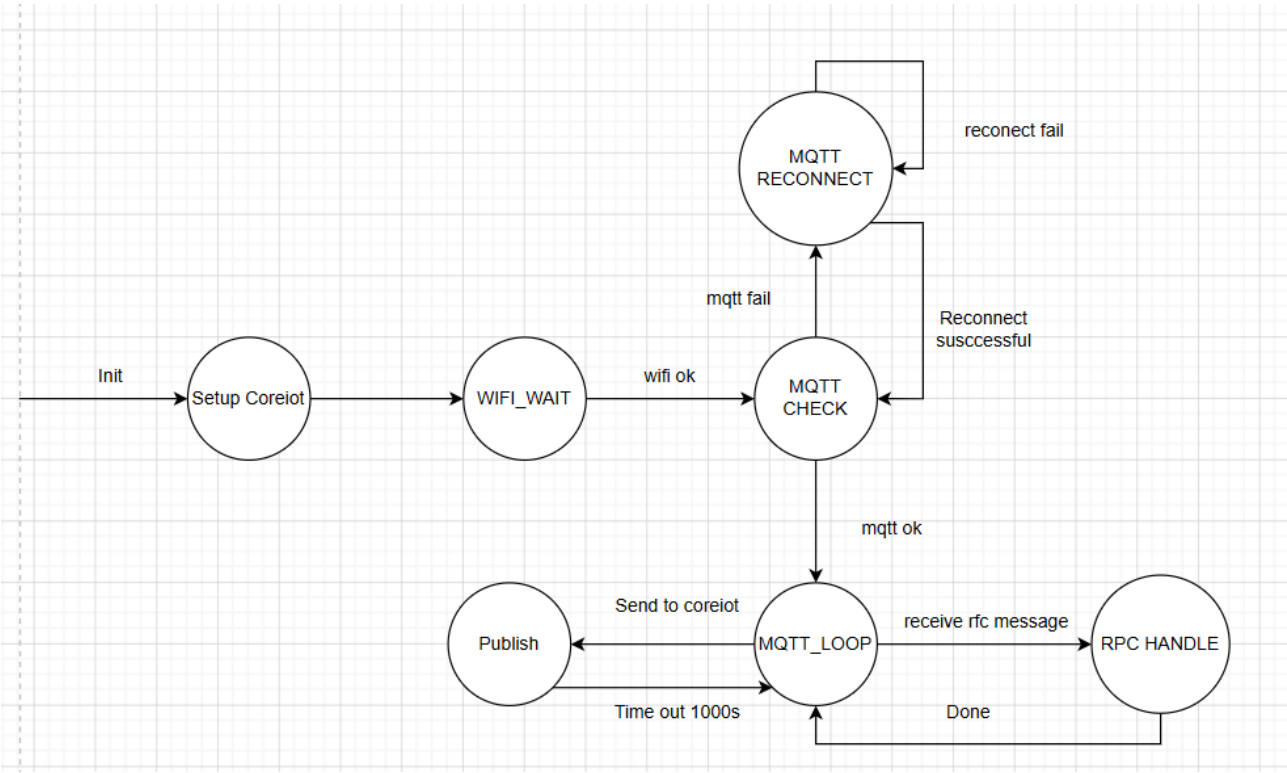
[Mô tả định dạng dữ liệu gửi lên đám mây, quy trình publish và chiến lược đảm bảo độ tin cậy/thử lại.]

3.7.1 Nguyên lý hoạt động

Tác vụ "CoreIoT"đóng vai trò là cầu nối giao tiếp hai chiều (Bidirectional Communication) giữa biên thiết bị (ESP32) và nền tảng đám mây (Cloud/ThingsBoard) thông qua giao thức MQTT ở port 1883. Tác vụ này thực hiện hai nhiệm vụ cốt lõi: báo cáo dữ liệu định kỳ và lắng nghe, thực thi các lệnh gọi hàm từ xa (RPC - Remote Procedure Call) từ người dùng.

- **Quản lý kết nối không xung đột:** Để tránh hiện tượng "tranh chấp"module Wi-Fi với Web Server gây treo hệ thống, Task CoreIoT không tự gọi hàm `WiFi.begin()`. Thay vào đó, nó liên tục giám sát cờ `isWifiConnected` do `main_server_task` cấp phát. Nếu mất Wi-Fi, tác vụ tự động chuyển sang trạng thái chờ 2000ms; chỉ khi có mạng, quá trình kết nối MQTT (`reconnect`) mới được phép kích hoạt.
- **Đồng bộ:** Khi duy trì kết nối MQTT ổn định, cứ mỗi 10.000ms (10 giây), hệ thống sẽ đóng gói các thông số môi trường hiện hành (`glob_temperature`, `glob_humidity`) và cấu hình ngưỡng cảnh báo (`glob_temp_threshold`) thành chuỗi JSON. Chuỗi này được gửi lên topic `v1/devices/me/telemetry` để hiển thị lên Dashboard của hệ thống CoreIoT.
- **Thực thi lệnh điều khiển RPC (RPC Callback):** Hệ thống đăng ký (subscribe) lắng nghe tại topic `v1/devices/me/rpc/request/+`. Khi có lệnh điều khiển từ Dashboard (VD: kéo Slider thay đổi ngưỡng), hàm `callback` sẽ được ngắt để gọi. Thư viện `ArduinoJson` phân tích gói tin, nhận diện phương thức `"setThre"` và ngay lập tức trích xuất tham số `params` để cập nhật đè lên biến `glob_temp_threshold`. Nhờ đó, thay đổi từ Cloud lập tức được phản ánh xuống giới hạn cảnh báo của màn hình LCD ở biên thiết bị.

3.7.2 Sơ đồ máy trạng thái (FSM)



Hình 4: Task 6 FSM

3.8 Tổng quan kiến trúc

[Trình bày kiến trúc tổng thể của hệ thống, luồng dữ liệu và sự tương tác giữa các tác vụ.]

3.9 Tính mới và cải tiến

[Nêu bật các tính năng sáng tạo, nâng cấp giao diện và các mở rộng TinyML để tăng điểm cộng.]

3.10 Tóm tắt chương

[Tóm tắt nội dung chương triển khai.]

4 Đánh giá và kết quả thực nghiệm

4.1 Giới thiệu

[Giới thiệu chương đánh giá và các thước đo được sử dụng.]

4.2 Các kịch bản kiểm thử

[Mô tả các kịch bản kiểm thử: mạng yếu, mạng không ổn định và các điều kiện cảm biến thay đổi.]

4.3 Kết quả chức năng

[Trình bày tính đúng đắn và giải thích các hành vi chức năng quan sát được.]

4.4 Đánh giá TinyML

[Đánh giá độ chính xác, độ trễ và thời gian phản hồi của pipeline TinyML.]

4.5 Thảo luận

[Thảo luận về điểm mạnh, điểm nghẽn và các đánh đổi quan sát được trong thực nghiệm.]

4.6 Tóm tắt chương

[Tóm tắt nội dung chương đánh giá.]

5 Kết luận và thảo luận nhóm

5.1 Thảo luận và kết luận

[Tổng kết quá trình triển khai, các thách thức, bài học kinh nghiệm và kết luận cuối cùng.]

5.2 Hướng phát triển trong tương lai

[Đề xuất các cải tiến trong tương lai hoặc một hướng triển khai lại, ví dụ MicroPython.]

Tài liệu tham khảo

Tài liệu

[1] [Bổ sung tài liệu tham khảo 1]

[2] [Bổ sung tài liệu tham khảo 2]

A Phụ lục A: Tổ chức nhóm

[Bổ sung vai trò từng thành viên, trách nhiệm và thông tin đóng góp Git tại đây.]

B Phụ lục B: Tài liệu bổ sung

[Bổ sung sơ đồ khối, trích đoạn mã nguồn và các kết quả phụ trợ tại đây.]